

Einschränkungen von Serverless Compute

Serverless Compute vereinfacht vieles, hat aber im Vergleich zu Classic Compute auch einige technische Einschränkungen. Diese Seite listet die wichtigsten Limitierungen auf, damit man vorab weiß, ob Serverless Compute für den eigenen Anwendungsfall geeignet ist.

Sprachen und APIs

- Die Programmiersprache R wird nicht unterstützt.
- Nur Spark-Connect-APIs stehen zur Verfügung, die klassischen Spark-RDD-APIs nicht (RDDs sind die "rohe" Low-Level-Datenstruktur von Spark).
- Spark Connect wertet Code erst zur Laufzeit aus ("deferred analysis"), was das Verhalten von Code im Vergleich zu klassischem Spark verändern kann.
- ANSI-SQL ist standardmäßig aktiviert; man kann dies über `spark.sql.ansi.enabled` deaktivieren.
- Beim Erstellen von DataFrames aus lokalen Daten dürfen einzelne Zeilen nicht größer als 128 MB sein.

Datenzugriff und Speicher

- Für den Zugriff auf externe Datenquellen ist Unity Catalog zwingend erforderlich.
- Der Zugriff auf DBFS (Databricks File System) ist eingeschränkt; stattdessen werden Volumes oder Workspace-Dateien empfohlen.
- Maven-Koordinaten (zum Einbinden von Java-/Scala-Bibliotheken) werden nicht unterstützt.
- Globale temporäre Views sind nicht erlaubt; stattdessen sollten sitzungsbezogene temporäre Views oder Tabellen verwendet werden.
- DBFS-Mounts über AWS-Instanzprofile werden nicht unterstützt.

User-Defined Functions (UDFs)

- UDFs (benutzerdefinierte Funktionen) haben keinen Internetzugriff; das Erstellen externer Verbindungen aus UDFs heraus ist untersagt.

- Benutzerdefinierter Code wie UDFs, `map` und `mapPartitions` darf nicht mehr als 1 GB Arbeitsspeicher verbrauchen.
- Scala-UDFs können nicht innerhalb von High-Order-Functions verwendet werden.

Benutzeroberfläche und Logging

- Die klassische Spark-UI ist nicht verfügbar; als Alternative dient das Query Profile.
- Nur clientseitige Anwendungs-Logs sind zugänglich, Spark-Logs selbst nicht.

Netzwerk und Workspace-Zugriff

- Der Zugriff auf andere Workspaces setzt voraus, dass diese in derselben Region liegen, ohne IP-Zugriffslisten (IP-ACLs) oder Front-End-PrivateLink.
- Databricks Container Services werden nicht unterstützt.

Streaming-Einschränkungen

- Unterstützte Trigger: `Trigger.AvailableNow()` (empfohlen) und `Trigger.Once()` (veraltet).
- Nicht unterstützt: `Trigger.Continuous()` und `Trigger.ProcessingTime()`.
- Der Standard-Trigger muss bei Serverless Compute explizit überschrieben werden.
- Für dauerhaftes ("continuous") Streaming sind alternative Ansätze bzw. Modi nötig.
- Alle allgemeinen Streaming-Einschränkungen des Standard-Access-Modes gelten zusätzlich.

Einschränkungen bei Notebooks

- Scala und R werden nicht unterstützt.
- JAR-Bibliotheken sind in Notebooks nicht nutzbar (JAR-Tasks in Jobs funktionieren hingegen).
- Notebook-spezifische Bibliotheken werden nicht über mehrere Entwicklungssitzungen hinweg zwischengespeichert.
- Temporäre Tabellen und Views können nicht zwischen verschiedenen Nutzern eines Notebooks geteilt werden.
- DataFrame-Autovervollständigung und der Variable Explorer stehen nicht zur Verfügung.
- Notebooks verwenden standardmäßig das .ipynb-Format; Notebooks im Source-Format können dabei Metadaten verlieren.
- Notebook-Tags werden nicht unterstützt; stattdessen dienen Serverless Usage Policies zur Kennzeichnung.

Einschränkungen bei Jobs

- Task-Logs sind nicht pro individuellem Task-Lauf isoliert.
- Task-Bibliotheken werden für Notebook-Tasks nicht unterstützt; stattdessen sind notebook-spezifische Bibliotheken erforderlich.
- Standardmäßig gibt es kein Timeout für Abfragen; über `spark.databricks.execution.timeout` lässt sich eines konfigurieren.
- Die maximale Laufzeit beträgt 7 Tage – längere Läufe werden von der Plattform automatisch beendet.

Compute-spezifische Einschränkungen

- Compute-Policies, Init-Skripte und gezielt zugewiesene ("scoped") Bibliotheken sind nicht erlaubt.
- Instance Pools und Compute-Event-Logs stehen nicht zur Verfügung.
- Die meisten klassischen Spark-Compute-Konfigurationen werden nicht unterstützt.
- Umgebungsvariablen werden nicht unterstützt; stattdessen werden Widgets und Job-Parameter empfohlen.

Caching-Einschränkungen

- Durch das Zwischenspeichern von Sitzungsmetadaten wird beim Wechsel des Katalogs unter Umständen kein vollständiger Kontext-Reset durchgeführt.
- DataFrame- und SQL-Cache-APIs sind untersagt, darunter `df.cache()`, `df.persist()` und `CACHE TABLE`.

Hive-Einschränkungen

- Hive-SerDe-Tabellen sowie der zugehörige `LOAD DATA`-Befehl werden nicht unterstützt.
- Unterstützte Datenquellenformate sind beschränkt auf: AVRO, BINARYFILE, CSV, DELTA, JSON, KAFKA, ORC, PARQUET, TEXT und XML.
- Hive-Variablen und Konfigurationsvariablen-Referenzen sind untersagt; stattdessen sollten SQL-Sitzungsvariablen und Parameter-Platzhalter verwendet werden.

Unterstützte Datenquellen im Überblick

Kategorie	Unterstützte Formate/Quellen
Lese- und Schreibzugriff (DML)	CSV, JSON, AVRO, DELTA, KAFKA, PARQUET, ORC, TEXT, UNITY_CATALOG, BINARYFILE, XML, SIMPLESCAN, ICEBERG
Nur Lesezugriff	Zusätzlich MYSQL, POSTGRESQL, SQLSERVER, REDSHIFT, SNOWFLAKE, SQLDW, DATABRICKS, BIGQUERY, ORACLE, SALESFORCE, SALESFORCE_DATA_CLOUD, TERADATA, WORKDAY_RAAS, MONGODB

Quelle: <https://docs.databricks.com/aws/en/compute/serverless/limitations> · Stand: 2026-08-06